

MITs: Mobility-aware Intelligent Task Scheduling in Vehicular Fog Networks

Mekala Ratna Raju, Sai Krishna Mothku, *Member, IEEE*, and Manoj Kumar Somesula, *Member, IEEE*,

Abstract—Vehicular Fog Computing (VFC) has emerged as a captivating research paradigm to increase computing resources for Internet of Vehicles (IoV) applications. However, the allocation of the best possible computing resources is challenging due to the enormous growth of services in vehicular networks, such as augmented reality, 3-D gaming, autonomous driving, etc., and their severe delay limitations, processing demands, and mobility of vehicles. Mainly, achieving strict task latency is a challenging problem while vehicles traverse through the roadside unit (RSU) regions. In this work, the vehicle's task processing latency and energy consumption are considered by deciding which tasks should be scheduled to either the local RSU or the neighbour RSUs along the vehicle's path. We study the mobility-aware task scheduling problem to jointly optimize the service latency of the tasks and energy consumption of the vehicles. Initially, we employ the Markov renewal process (MRP) to model the vehicle's mobility. Subsequently, we establish that our proposed objective function is a monotone submodular function and provide a greedy heuristic for the maximization of the submodular function with an approximation of $(1 - \frac{1}{e})$. Further, we propose a hybrid optimization algorithm that combines particle swarm optimization (PSO) and simulated annealing (SA) algorithms to solve mobility-aware task scheduling problems for large-scale networks. We compare our proposed algorithm with existing mobility-aware scheduling approaches and simulation results show that the proposed algorithm performs efficiently for task scheduling problems.

Keywords—Internet of Vehicles, Vehicular Fog Computing, Task offloading, Task scheduling, Response time, Energy consumption.

I. INTRODUCTION

With the proliferation of the Internet of Vehicles (IoV), intelligent vehicles offer more computation-intensive and delay-sensitive applications, including driving assistance, traffic monitoring, autonomous administration, and entertainment [1]. Nowadays, vehicles' application data and requirements are increasing exponentially and becoming latency-sensitive and computation intensive [2]. The vehicles may not be able to support large-scale applications due to their limited computing and battery capacity. This makes it more challenging for the vehicles to ensure the quality of the services [3].

Cloud computing emerged as the IoV's potential paradigm to handle the enormous requests and expectations of the IoV. Vehicles offload their computationally demanding requests through wireless networks to the cloud's high-capacity data

centres. The standard cloud data centres are located far from the vehicles, making it difficult to offload the vehicle requests for delay-sensitive applications. Hence, the transmission of the IoV requests to cloud servers leads to high network delays and severe deterioration in the quality of the service. Fog computing has emerged as a potential paradigm to provide services for IoT-enabled devices and vehicles to address the problems of cloud computing [4]. Fog computing can reduce transmission latency and improve the quality of service. Furthermore, as the use case of fog computing, vehicular fog computing (VFC) has recently gained attention as a new computing field in the domain of intelligent transportation systems. VFC extends the cloud computing capability to the edge of the vehicles to provide IoV services through roadside units (RSU) [5].

In a vehicular fog computing environment, one of the critical challenges is the scheduling of IoV tasks. A vehicle generates an independent number of requests and offloads to the corresponding RSU server to execute them. The inappropriate scheduling of vehicular tasks decreases the efficiency of VFC networks. IoV task scheduling has been extensively studied, but few studies have been conducted on vehicle mobility and sojourn time. In vehicular fog networks, prior knowledge of the vehicle's mobility patterns enhances the efficiency of task scheduling. The vehicle may move from one RSU region to another RSU region after transmitting the requests to the corresponding RSU server. This results in the task requests failing, and the vehicle must again transmit the same requests to the new RSU. Nevertheless, executed task results will be forwarded to the cloud, and from the cloud server, the vehicle gets the task result, which is time-consuming.

The above-mentioned challenges motivated us to investigate a new mobility-aware task scheduling problem in vehicular fog computing architecture, where the vehicles transfer tasks to RSU for execution. The base station decides how many tasks of the vehicles should be scheduled in the vehicle and how many should be scheduled for the available RSU servers in the vehicle path. The existing research investigates the mobility-aware offloading strategies based on the velocities, single path uni-directional or considering the mobility only in the single RSU [6, 7]. However, in a vehicular fog network, a vehicle can travel at random velocities based on the traffic, and it can travel not only a single path uni-directional but also in multipath multi-direction. Therefore, to achieve efficient task scheduling, it is necessary to determine the dwell time and path of vehicles within the base station region. The predicted mobility information plays a vital role in effectively utilizing the available resources.

The requests of vehicular applications involve massive computations, which use more energy in the processors of the

M.R.Raju and S.K.Mothku are with the Department of Computer Science and Engineering, National Institute of Technology Tiruchirappalli-620015 (Email: ratnarajumeekala@gmail.com and saikrishna@nitt.edu).

M.K.Somesula is with the Department of Computer Science and Engineering, National Institute of Technology Jalandhar-144011 (Email: kumarsm@nitj.ac.in).

vehicles. Moreover, when tasks are offloaded to the RSU servers, it incurs more delay due to the limited processing capacities of RSU servers. Therefore, it is crucial to construct a delay-aware and energy-efficient task allocation scheme to cope with IoV applications. In this paper, we address the flaws of existing VFC task scheduling strategies by considering the deadline of the tasks and mobility information of the vehicles to minimize the energy consumption of vehicles and the latency of vehicular computation tasks. Vehicle mobility is formulated as a Markov renewal process (MRP) to predict the vehicles' sojourn time and moving paths. We then prove the objective function of the proposed task scheduling problem as a submodular function and provide a greedy solution for maximizing the function. However, the greedy solution is efficient for small-scale vehicular fog networks. Hence, we propose a hybrid optimization approach using features of particle swarm optimization (PSO) and simulated annealing (SA) for the mobility-aware task scheduling problem to cope with large-scale networks.

The following are our major contributions:

- A mixed integer non-linear programming (MINLP) problem is formulated for the mobility-aware task scheduling problem with the objective of joint optimization of latency of the tasks and energy consumption of the vehicles.
- A Markov renewal process (MRP) is presented for the vehicle's mobility model to predict the vehicle travelling path and sojourn time.
- We reformulate the objective function as sub modular optimization function and provide a greedy solution for the problem.
- We provide a hybrid optimization algorithm which combines particle swarm optimization (PSO) and simulated annealing (SA) for the mobility-aware task scheduling problem for large-scale networks.
- We evaluate the performance of the proposed algorithm with existing state-of-the-art algorithms by performing extensive experiments.

The rest of the paper is organized as follows. Section II summarizes the review of the related works on task scheduling in the vehicular fog computing environment. The system model of the vehicular fog computing environment is presented in section III, along with problem formulation. Section IV presents the greedy algorithm for the proposed mobility-aware task scheduling problem. Section V illustrates the simulated annealing-based particle swarm optimization algorithm for the proposed problem. Experiment results are discussed in Section VI, and Section VII concludes the paper.

II. RELATED WORK

In recent years, researchers have focused on scheduling tasks in fog computing. Task scheduling within the Internet of Vehicles has gained substantial attention as a prominent research area in the modern digital world [8]. We have divided the related work of task scheduling into four categories: i) latency-aware scheduling, ii) energy-aware scheduling, iii) latency and energy-aware scheduling, and iv) mobility-aware scheduling, which are presented as follows.

A. Latency Aware Scheduling

In a vehicular fog network, reducing the service delay of the tasks is essential to avoid deadline miss and to satisfy the user. In [9], Zhang *et al.* have investigated a resource allocation problem in vehicular fog networks to reduce the transmission time using the weighted minimum mean error method and Perron-Frobenius theory. Liu *et al.* have designed a task offloading scheme [10] using matching theory to reduce the total network latency in a vehicular edge computing environment. In [11], Sun *et al.* have developed a task scheduling scheme in vehicular edge networks for the reduction of the total processing delay of the tasks and resource consumption using a hybrid partheno genetic algorithm.

Wang *et al.* [12] have designed a game-based task offloading strategy to maximize the utility of the vehicles, which minimizes the total processing delay of the tasks using a distributed algorithm in a vehicular edge computing environment. In [13], Zhou *et al.* have designed a task allocation mechanism to assign tasks for the minimization of network delay in a vehicular fog computing environment using contract matching theory.

B. Energy Aware Scheduling

In the scheduling process of vehicular tasks, the vehicle consumes its energy during communication and computation. Hence, minimizing the vehicle's energy usage during task scheduling is very important. In [14], Qin *et al.* have designed a task offloading technique further divided into three subproblems such as task splitting, offloading and resource assignment and, solved using Lyapunov optimization, matching theory and greedy approach, respectively, to minimize the energy consumption of the vehicles in vehicular fog networks. To reduce the energy usage of the user equipment in the vehicular edge networks, Zhou *et al.* [15] have proposed a resource allocation mechanism using the alternating direction method of multipliers (ADMM) technique.

Abdel-basset *et al.* in [16] have designed a task scheduling in a fog computing environment for the minimization of the energy consumption in the IoT-fog computing system using marine predators optimization algorithm. Zhai *et al.* in [17] have devised a task offloading approach in the SDN-enabled IoV framework to minimize the energy consumption of the edge devices using a heuristic optimization technique. In [18], Liu *et al.* have designed a task offloading strategy to minimize the energy consumption of vehicles in vehicular fog networks using a general heuristic algorithm. Hazra *et al.* in [19] have developed a task scheduling approach in industrial IoT applications using matching theory to minimize the energy consumption of the devices.

C. Delay and Energy Aware Scheduling

A vehicle has delay-sensitive tasks and limited battery capacity. Therefore, balancing the service delay of the tasks and energy consumption of the vehicle is challenging in VFC networks. In [6], Raza *et al.* have proposed a mobility-aware task offloading and resource allocation strategy to improve computation efficiency, including computation delay and energy consumption, using a game-theoretic approach. In [20],

Luo *et al.* have investigated a computation offloading mechanism to optimize the delay and energy in a vehicular edge computing environment using a particle swarm optimization technique. In [21], Tang *et al.* have developed a task offloading strategy in mobile edge networks to reduce the response time of all industrial tasks using a greedy algorithm. In [22], Chen *et al.* have proposed a task offloading mechanism in a mobile edge computing environment to offload the tasks in mobile edge servers for the minimization of the service time of tasks and energy consumption of the user equipment using game theory strategies.

D. Mobility Aware Scheduling

In a VFC environment, it is essential to consider the vehicle's mobility through the regions of RSU servers to improve the **efficiency of the task scheduling**. Lin *et al.* [23] have designed a mobility-aware task scheduling approach to assign tasks to available fog base station servers in a vehicle's moving path to minimize the processing time of the tasks in a vehicular fog computing environment using a general heuristic algorithm. In [7], Misra *et al.* have designed a mobility-aware task offloading strategy using a greedy heuristic to minimize the computing delay of the tasks in software define network-enabled vehicular fog networks. Lakhan *et al.* have designed a mobility aware task scheduling technique in [24] to minimize the communication and computation cost of the IoV applications using the general heuristic solution.

All the aforementioned papers do not focus on scheduling the vehicular tasks while considering the **vehicle's sojourn time and mobility path**. Since the vehicle leaves its corresponding RSU and joins another RSU region, scheduling of vehicular tasks in vehicular fog networks to reduce the delay and energy consumption is a cumbersome task. However, in our work, we considered the **mobility and sojourn time of the vehicle to allocate the tasks in an efficient manner**. To allocate the tasks in available servers, researchers have also adopted machine learning and deep learning techniques [25, 26]. However, they **consume more computation time to execute the algorithm and require huge amounts of data**. Hence, we have not considered deep learning techniques for resource-constrained servers in our proposed approach. The network model and proposed task-scheduling algorithms are presented in the following sections.

III. SYSTEM MODEL AND PROBLEM FORMULATION

The system model of the proposed system and problem formulation are presented in this section.

A. Network Model

We consider an urban vehicular fog network scenario, as shown in Fig. 1, comprised of several Road Side Units (RSU) distributed and deployed near roads in a smart city further controlled by a base station. The RSUs are defined as a set $\mathcal{R} = \{1, 2, \dots, r, \dots, R\}$, where R is the number of RSUs, **and the coverage area of RSUs do not overlap**. Generally, the computing resources of the RSU can be represented in terms of the capabilities of the central processing unit (CPU), memory, and network interfaces [27]. For ease of analysis, we only consider the CPU capacity of the RSU server. We assume that an RSU r has a vehicular fog computing (VFC) server, which

TABLE I: Summary of Notations

Symbol	Description
$r, \mathcal{R}$	RSU r in set of RSUs $\mathcal{R}$
$v, \mathcal{V}$	Vehicle v in set of Vehicles $\mathcal{V}$
R	Total number of RSUs
V	Total number of Vehicles in an RSU region
$\mathcal{A}_v$	Set of tasks of application v
$\mathcal{A}_v^l$	Set of tasks to be executed in vehicle v
$\mathcal{A}_v^o$	Set of tasks to be executed in RSUs
s_v	Size of the application in bits
c_v	Number of cycles required for application v
d_v	deadline of the application v
$x_{v,i}^r$	Decision to variable to indicate the i^{th} task of the application v allocated to the server r
B_{vr}^r	Bandwidth of the channel between the vehicle v and RSU r
ρ_{vr}^r	Transmission rate of the channel between the vehicle v and RSU r
P_v^r	Transmission power of the vehicle v
G_{vr}^r	Channel Gain of the channel between vehicle v and RSU r
t_v^m	Transmission time of the vehicle v to upload its application A_v
f_r	Computation capacity of the r^{th} RSU server
X_g	Transition state g
T_g	Time instant of transition state g
Δ_v^r	Sojourn time of the vehicle v in RSU r region
P_v	The transition probability of Markov Chain for vehicle v
H_v^r	The probability distribution of time that vehicle v dwells in RSU region r
$\mathcal{R}^{max}$	The maximum number of resources available in the VFC system
E_v^{avl}	Energy available in the vehicle v

is defined by (rid, f_r) , where rid represents the id of the RSU server r and f_r represents the processing capacity of the RSU server (cycles/sec). We also consider the internet of vehicular (IoV) applications, defined as a set $\mathcal{V} = \{1, 2, \dots, v, \dots, V\}$, where V represents the number of vehicular applications. **We consider that each vehicle v has one application at a time**. For ease of understanding, we employ v to represent the vehicle and its application. RSUs can provide the services to the IoV. Under the coverage area of RSU, vehicles can connect to the corresponding RSU. Each RSU has a fog server to process the requests of vehicles, and all RSUs are further connected to the base station. For ease of reference, we used the notations from Table I.

The system time is partitioned into fixed time slots. In each time slot of the system, vehicles send the requests to the base station for the execution of applications. The base station observes the requests from the vehicles for the execution. **Based on the resources of the RSU fog servers, vehicle capacity and mobility, the base station decides the allocation of application tasks to either the vehicle CPU or RSU servers in the vehicle's path**. The information of each vehicle's application is illustrated in section III-B. It is illustrated in Fig. 1 that each vehicle has **one application which is further divided into independent tasks**. From the figure, it is observed that the blue colour vehicle $v1$ has an application which contains ten independent sub tasks and requests the base station for the allocation decision, the base station allocates the three tasks in RSU server-1, five tasks in RSU server-2, and the remaining two tasks in RSU server-3 based on the vehicles mobility, sojourn time, communication and computation resources.

B. Task Model

In this section, we introduce the task model of the IoV application. Each application can be defined as $A_v = \{s_v, c_v, d_v\}, v \in V$. Where s_v represents the data size of the application in bits, c_v represents the required number

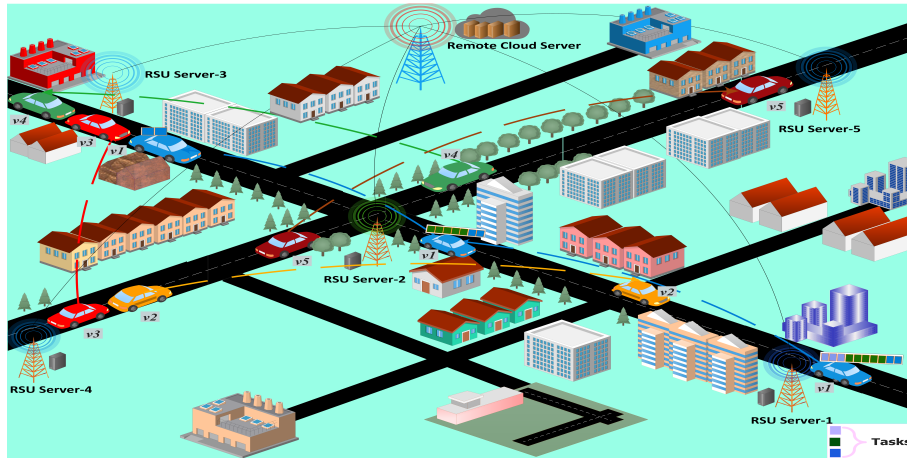


Fig. 1: Fog enabled IoV Framework

of cycles to complete the application, and d_v represents the deadline of the application. **We consider that each application can be divided into a set of independent tasks which can be allocated separately to the servers. For example, in an autonomous driving scenario, a set of images can be separated into several subsets that can be processed in parallel** [18]. Each task of the application can be allocated to either vehicle or RSU servers based on the vehicle's mobility. The divided set of tasks of an application v is denoted as $\mathcal{A}_v = \{A_{v,1}, A_{v,2}, \dots, A_{v,i}, \dots, A_{v,n}\}$, where n represents the number of tasks of application v and the i^{th} task of v^{th} vehicle application can be represented as $A_{v,i}$. To ensure that IoV tasks are offloaded and executed in the vehicles and RSU servers, we first define the decision variable $x_{v,i}^r$.

$$x_{v,i}^r = \begin{cases} 1, & \text{if task } i \text{ of } v^{th} \text{ application is assigned to server } r \\ & \text{either vehicular processor or RSU server} \\ 0, & \text{otherwise} \end{cases} \quad (1)$$

Where r in $x_{v,i}^r$ indicates both the vehicle and RSU servers. We have separated the tasks that are executed locally in the vehicle are represented by $\mathcal{A}_v^l$ and tasks that are to be offloaded to RSU servers are represented by $\mathcal{A}_v^o$ based on the mobility of the vehicle i.e., $\mathcal{A}_v = \mathcal{A}_v^l \cup \mathcal{A}_v^o$.

C. Mobility Model

Vehicles send the request to the RSU and move from one RSU coverage to another RSU coverage in a specific time slot. The vehicle's mobility is modelled as the Markov renewal process ($\{(X_g, T_g) : g \geq 0\}$) to predict the path of the IoVs and their dwell time at RSUs, where X_g is the g^{th} transition state, and T_g is the g^{th} transition time instant. The time duration of a vehicle v is served by an RSU r is called sojourn time, and it is obtained by the vehicle moving statistics and is denoted as Δ_v^r . The transition probability of the Markov chain for vehicle v is P_v . H_v^r is the distribution of time that the semi-Markov process of vehicle v dwells at RSU r coverage before the vehicle is moved to another RSU.

1) *Mobility Prediction:* During a time period T , a vehicle v travels from one state to another. It is unknown how long a vehicle stays in the RSU r . The sojourn duration of the vehicle

Δ_v^r dwells in RSU r is predicted using the Markov renewal process [28, 29]. The average time is computed to predict the sojourn time, which is given as follows:

$$\mu_v^r = \int_0^\infty y H_v^r(y) dy \quad (2)$$

$H_v^r(y)$ denotes the probability density function for the sojourn time of the vehicle v in a state r , i.e., the vehicle is expected to travel from one RSU to another by staying μ_v^r average time duration in RSU r .

Vehicle may travel in various directions. The vehicle v travelling along a path in a time period T is represented as $\mathbf{Z} = \{z_v^1, z_v^2, \dots, z_v^k\}$, where $k \geq 1$. The starting state of the path k is denoted as $z_v^{k,0} = z_v^k$, i.e., the starting state of the path k is the starting state of all paths of the vehicle v . Hence, the k^{th} path of vehicle v is represented as $z_v^k = \{z_v^{k,0}, z_v^{k,1}, \dots, z_v^{k,b}\}$, where $b \geq 0$ and $z_v^{k,1}$ denotes the k^{th} path of the vehicle v at first RSU. A vehicle may return to the same RSU region several times in a certain path. Hence, the vehicle's path z_v^k is a multi-set. The vehicle dwells in the initial state for t_v^0 and takes the first transition. For all the paths, the first transition is expected to take place in time instant as follows:

$$T_v^{k,1} = \int_0^\infty y H_v^{z_v^{k,0}}(y) dy \quad (3)$$

Where $T_v^{k,0}$ is 0 and $T_v^{k,1}$ is constant for all the paths of the vehicle v . Therefore, the k^{th} path j^{th} transition is expected to be happened as follows:

$$T_v^{k,j} = T_v^{k,j-1} + \mu_v^{z_v^{k,j-1}} \quad (4)$$

where $j \leq b$. The final transition occurs at the end of time instant T .

The sojourn time of the k^{th} path z_v^k is represented as $\Delta_v^k = \{\Delta_v^{z_v^{k,0}}, \Delta_v^{z_v^{k,1}}, \dots, \Delta_v^{z_v^{k,b}}\}$. The sojourn time is $\Delta_v^{z_v^{k,j}} = T_v^{k,j} - T_v^{k,j-1}$. Hence, the sojourn time is elaborated as:

$$\Delta_v^{z_v^{k,j}} = \begin{cases} T_v^{k,1}, & j = 0 \\ [\mu_{z_v^{k,j}}^r], & 1 \leq j \leq b-1 \\ [T - \sum_{w=0}^{b-1} \Delta_v^{z_v^{k,w}}], & j = b \end{cases} \quad (5)$$

Where $q = [u]$ denotes the value $q = 0$ when $u < 0$ and $q = u$ when $u \geq 0$. The probability of a vehicle v choosing the path z_v^k is defined as:

$$P_v^k = \prod_{j=0}^{b-1} P_{z_v^{k,j}, z_v^{k,j+1}} \quad (6)$$

Therefore, The average sojourn duration of vehicle v at RSU r during the time period T is denoted as:

$$\Delta_v^r = \sum_{k \in K} P_v^k \sum_{w=0}^d \Delta_v^{z_v^{k,w}} 1_{(z_v^{k,w}=r)}, \forall r \in \mathcal{R} \quad (7)$$

D. Communication Model

In this section, we present the communication model of the tasks to the RSUs server. We assume that the communication channel between IoV and RSU is wireless. Similar to the [30], we assume that the wireless communication between the IoVs and RSUs follows the **orthogonal frequency division multiple access mechanism**. The data transmission rate of the channel between vehicle v and RSU r , ρ_v^r is defined as follows:

$$\rho_v^r = B_v^r \log \left(1 + \frac{P_v^r G_v^r}{N_0} \right) \quad (8)$$

Where B_v^r indicates the bandwidth of the channel between the vehicle v and RSU r , P_v^r denotes the transmission power of the vehicle v , G_v^r denotes the channel gain between vehicle v and RSU r , and N_0 is the noise power. Hence, the transmission delay taken by the vehicle v for offloading its task i to RSU r is defined as follows:

$$tm_{v,i}^r = \frac{s_{v,i}}{\rho_v^r}, \forall i \in A_v^o, \forall r \in \mathcal{R} \quad (9)$$

Where A_v^o represents the set of tasks to be offloaded to RSUs. $s_{v,i}$ indicates the size of the i^{th} task of the v^{th} application, **which includes both input and output data of the tasks. We assume that the transmission rate of the downlink is the same as the transmission rate of the uplink.** Therefore the total transmission time to offload the tasks of vehicle v to RSU r is defined as:

$$tm_v^r = \sum_{i \in A_v^o} \sum_{r \in \mathcal{R}} x_{v,i}^r * \frac{s_{v,i}}{\rho_v^r}, \forall v \in \mathcal{V} \quad (10)$$

Hence, the energy consumed to offload the tasks of the application to the corresponding RSU server is defined as follows:

$$E_v^c = P_v^r * \sum_{i \in A_v^o} \sum_{r \in \mathcal{R}} x_{v,i}^r * \frac{s_{v,i}}{\rho_v^r}, \forall v \in \mathcal{V}, \quad (11)$$

Where A_v^o represents the set of tasks of v^{th} application which is allocated to RSUs. ρ_v^r represents the transmission rate of the channel between the RSU r and vehicle v . P_v^r represents the transmission power of channel between RSU r and vehicle v .

E. Computation Model

The computation model of the system to execute the IoV applications in the RSU server is presented in this section. We consider that the resources of the RSU server are limited, and each RSU server can process only one task at any instant.

1) *Local Computing*: Initially, the IoV application requests are transmitted to the base station. The base station decides the tasks to be executed in the vehicle and those to be offloaded

to RSU fog servers. The computation time to execute the task i of an application v scheduled for the vehicle is as follows:

$$t_{v,i}^l = \frac{c_{v,i}}{f_v}, \forall i \in v \quad (12)$$

Where $c_{v,i}$ represents the required number of cycles to execute the i^{th} task of v^{th} application. f_v represents the computation capacity of the v^{th} vehicle. Some of the tasks of the vehicle v are executed in the vehicle instead of allocating to RSUs. The total computation time to execute the tasks locally in vehicle v is calculated as:

$$t_v^l = \sum_{i \in A_v^l} \frac{c_{v,i}}{f_v}, \forall v \in \mathcal{V} \quad (13)$$

2) *RSU Computing*: The IoV applications are offloaded to RSU fog servers through wireless channels depending on the motion and dwell time of the vehicles. Then, the fog server with computation capability will process the requests of IoV and return the results to the vehicles. The base station decides the tasks to be executed in RSUs in the vehicle path. The total computation time to execute the tasks which are scheduled to RSU servers is defined as follows:

$$t_v^e = \sum_{i \in A_v^o} \sum_{r \in \mathcal{R}} \left(x_{v,i}^r * \frac{c_{v,i}}{f_r} \right), \forall v \in \mathcal{V} \quad (14)$$

Where, $c_{v,i}$ represents the required number of cycles to execute the i^{th} task of v^{th} application. f_r represents the computation capacity of r^{th} server.

Hence, the total computation time to process the application A_v is defined as follows:

$$t_v^e = \max(t_v^l, t_v^e) \quad (15)$$

We consider the energy consumed to execute the tasks of applications which are allocated in vehicle processor that is given below:

$$E_v^l = \sum_{i \in A_v^l} k f_v^2 * c_{v,i}, \forall v \in \mathcal{V} \quad (16)$$

Where A_v^l represents the set of tasks of v^{th} application and k represents the coefficient which depends on the vehicle onboard processor chip capacitance [31].

F. Problem Formulation

The total service time of the task i of the application v is the sum of communication time and computation time, which is defined as follows:

$$t_{v,i} = tm_{v,i}^r + tw_{v,i}^r + t_{v,i}^e \quad (17)$$

Where $tm_{v,i}^r$ denotes the transmission time of the task i of vehicle v , if the task is locally scheduled in the vehicle, then $tm_{v,i}^r = 0$. And $tw_{v,i}^r$ includes both the sojourn time of the vehicle v in previous RSUs and the waiting time of the i^{th} task of application v in the allocated server r , which is the difference between the beginning of the execution time and the arrival time of the task. If the task is allocated to the vehicle v , then the waiting time of the task i does not include the sojourn time. $t_{v,i}^e$ denotes the time taken to execute the task i of the application v . $t_{v,i}^e$ is referred to either executed in the vehicle v or executed in RSU r . The total energy consumption of the vehicle to execute the tasks of the application is the sum of the transmission energy of the offloaded tasks, and the

computation energy of the local tasks is defined as follows:

$$E_v = E_v^c + E_v^l, \forall v \in \mathcal{V} \quad (18)$$

Our aim is to optimize the service delay of the tasks of the vehicles' applications and the vehicles' energy consumption in a fixed time slot under delay and resource constraints. The delay-energy trade-off is investigated, and the joint utility function is defined as follows:

$$\mathcal{U}(v, i) = \frac{1}{\lambda^t t_{v,i} + \lambda^E E_{v,i}}, \forall i \in v \quad (19)$$

Where $t_{v,i}$ is the service time of the task i of the vehicle v , and $E_{v,i}$ indicates the energy consumption of the vehicle to process the task i , if it is offloaded to RSU r , then transmission energy $E_{v,i}^c$ is considered, otherwise energy consumption of vehicle v to compute the task i , i.e., $E_{v,i}^l$ is considered. λ^t is the weight of the service time for each application, and λ^E is the weight of the energy consumption for each vehicle. Eqn. 19 is motivated from the paper [32] according to the proposed system model. The values of the symbols λ^t , λ^E should be in the range of 0 to 1, constrained to $\lambda^t + \lambda^E = 1$. Based on the requirement of the user, the values can be in the range of 0 and 1. However, in our proposed model, the delay and energy are given equal priorities, i.e., the values of λ^t and λ^E are 0.5, and 0.5, respectively. Therefore, the utility function for all the tasks of the vehicles can be elaborated as follows:

$$\mathbf{U} = \sum_{v \in \mathcal{V}} \sum_{i \in v} \sum_{r \in \mathcal{R}} \mathcal{U}(v, i) \quad (20)$$

We aim to balance the trade-off between the service delay of the tasks and energy consumption of the vehicles, which is the maximization of the utility function, which is as follows:

$$\text{Max. } \mathbf{U} \quad (21)$$

$$\text{s.t. } t_{v,i} \leq dl_{v,i}, \forall i \quad (22)$$

$$\sum_{i \in A_v^r} x_{v,i}^r * t_{v,i} \leq \Delta_v^r, \forall r \in \mathcal{R} \quad (23)$$

$$\sum_{r \in \mathcal{R}} x_{v,i}^r \leq 1, \forall v \in \mathcal{V} \quad (24)$$

$$\sum_{v \in \mathcal{V}} \sum_{i \in A_v^r} x_{v,i}^r * f_r \leq \mathcal{R}^{max}, \forall r \in \mathcal{R}, \forall v \in \mathcal{V} \quad (25)$$

$$E_v^c + E_v^l \leq E_v^{avl}, \forall v \in \mathcal{V} \quad (26)$$

$$|V_r| \leq C_r, \forall r \in \mathcal{R} \quad (27)$$

$$x_{v,i}^r \in \{0, 1\}, \forall r \in \mathcal{R}, \forall v \in \mathcal{V} \quad (28)$$

$$\lambda^t + \lambda^E = 1 \quad (29)$$

The objective function is to allocate the tasks of the applications of each vehicle to the resources available locally or in RSUs to maximize the utility function $\mathbf{U}$. Constraint 22 represents that the service time of the vehicle's task should not exceed the deadline. Constraint 23 implies that the service time of the vehicle requests should not exceed the sojourn time of the vehicle. Constraint 24 represents the task that should be allocated to only one server, whether it is a vehicle or RSU. Constraint 25 indicates that allocated resources for the tasks in an RSU should not exceed the available RSU resources. Constraint 26 indicates that the required energy to offload

and execute the tasks of the vehicle should not exceed the available energy of the vehicle. Constraint 27 indicates that the number of vehicles connected to RSU should not exceed the number of channels of RSU. Constraint 28 represents the decision variable, and constraint 29 shows the sum of the weights of the application's service time and the vehicle's energy consumption is 1.

Theorem 3.1: The objective function in Eqn. 21 is NP-hard problem.

Proof: For proof, refer to appendix A. ■

The task scheduling problem presented in Eqn. 21 is a mixed integer non linear programming (MINLP) problem and proved as an NP-hard problem [32]. The formulated objective function is proved as a monotone submodular function which is presented in the following sub-section. The maximization of the submodular function can be resolved using the greedy heuristic in the 0.63 approximation [33]. Hence, we have provided the greedy solution for maximizing the submodular problem in the following section.

IV. SUB MODULAR OPTIMIZATION

A. Preliminaries

We present the basic definitions of submodular function and matroid.

1) *Sub Modular Function:* Let g be a set function defined on the ground set $\mathcal{N}$, i.e., $g : 2^{\mathcal{N}} \rightarrow \mathbb{R}$. The function g is submodular if the following conditions are satisfied.

$$g(X \cup Y) + g(X \cap Y) \leq g(X) + g(Y), \forall X, Y \subseteq \mathcal{N} \quad (30)$$

$$g(X) \leq g(Y), \forall X \subseteq Y \subseteq \mathcal{N} \quad (31)$$

Let $g(X|j) = g(X \cup \{j\}) - g(X)$ be the marginal value of the function g with respect to the element j . In addition, g is said to be monotone if $g(X) \leq g(Y)$, for all $X \subseteq Y \subseteq \mathcal{N}$.

2) *Matroid:* A matroid is defined as $(\mathcal{N}, \mathcal{I})$, where $\mathcal{I} \subseteq 2^{\mathcal{N}}$ is a collection of sub sets of $\mathcal{N}$. Then $\mathbf{M}$ is a matroid if the following conditions are satisfied.

- $\mathcal{I} \neq \emptyset$
- If $X \subseteq Y, Y \in \mathcal{I}$, then $X \in \mathcal{I}$
- If $X, Y \in \mathcal{I}$, and $|X| < |Y|$, then there exist $e \in Y \setminus X$, such that $X \cup \{e\} \in \mathcal{I}$

We define the ground set $\mathcal{N}$, for the tasks assigned to the RSU servers, such as:

$$\mathcal{N} = \{x_{1,1}^0, \dots, x_{\mathcal{V},n}^0, x_{1,1}^1, \dots, x_{\mathcal{V},n}^1, x_{1,1}^{\mathcal{R}}, \dots, x_{\mathcal{V},n}^{\mathcal{R}}\}$$

Where $x_{v,i}^r$ implies that the allocation of the task i of the v^{th} application to the RSU server r . Where, $x_{v,i}^0$ indicates that the i^{th} task is assigned to the local server, i.e., v^{th} vehicle processor. The ground set $\mathcal{N}$ can be divided into $\mathcal{V}' = \mathcal{V} * \mathcal{R}$ disjoint sets i.e., $\mathcal{N} = \bigcup_{p=1}^{\mathcal{V}'} \mathcal{N}_p$, and $\mathcal{N}_p \cap \mathcal{N}_{p'} = \emptyset$, for any $p \neq p'$, where $\mathcal{N}_p = \{x_{v,1}^r, \dots, x_{v,n}^r\}$ with $p = \sum_{r'=1}^{r-1} (r' * \mathcal{V}) + v$. Let $\mathcal{N}_{p,i} = x_{v,i}^r$.

We also define a matroid $\mathcal{M} = (\mathcal{N}, \mathcal{I})$, where $\mathcal{N}$ is the

$$\begin{aligned}
 &\text{ground set, and } \mathcal{I} \subseteq 2^{\mathcal{N}} \text{ is the independent sets of } \mathcal{N}, \text{ i.e.,} \\
 &\mathcal{I} = \{X \subseteq \mathcal{N} : |X \cap \mathcal{N}_{p,i}| \leq 1, p = 1, 2, \dots, \mathcal{N}, \\
 &\quad t(X \cap \mathcal{N}_{p,i}) \leq dl_{v,i}, \\
 &\quad t(X \cap \mathcal{N}_p) \leq \Delta_v^r, \\
 &\quad |X \cap \mathcal{N}| \leq \mathcal{R}^{max}, \\
 &\quad E(X \cap \mathcal{N}_p) \leq E_v^{avl}, \\
 &\quad \sum_{p=1}^{\mathcal{V}'} |X \cap \mathcal{N}_{p,i}| \leq C_r, \forall r\}
 \end{aligned} \tag{32}$$

Where $X \cap \mathcal{N}_{p,i}$ represents the server allocated to the task i of the vehicle v . Where $t(X \cap \mathcal{N}_p)$ represents the service time of the tasks of vehicle v that are allocated to RSU r and $E(X \cap \mathcal{N}_p)$ represents the energy consumption of the vehicle v to process the task i . And we propose that the pair $(\mathcal{N}, \mathcal{I})$ is a matroid in the following lemma:

Lemma 4.1: The set family $M = (\mathcal{N}, \mathcal{I})$ is a matroid.

Proof: For proof, refer to appendix B. ■

In addition, based on Lemma 1, our proposed objective function is proved as a monotone submodular function in the following theorem.

Theorem 4.2: The objective function in the Eqn. 21 is monotone sub modular function over $X \in \mathcal{I}$

Proof: For proof, refer to appendix C. ■

By substituting the constraints in problem 21, with the matroid, $\mathcal{M} = (\mathcal{N}, \mathcal{I})$, the objective function can be reformulated as a maximization of monotone submodular function with matroid constraints is as follows:

$$\max_{X \in 2^{\mathcal{N}}} U(X) \tag{33}$$

$$s.t. X \in \mathcal{I} \tag{34}$$

A greedy algorithm is the best solution presented in the following section to solve the maximization of the monotone submodular function with matroid constraints.

B. Greedy algorithm for mobility aware task scheduling problem

The maximization of the submodular function presented in the Eqn. 33 can be efficiently solved with the greedy algorithm. In the worst case, the greedy task scheduling algorithm solves the problem in Eqn. 33 with an approximation of $(1 - \frac{1}{e})$. Therefore, it achieves a polynomial time complexity.

Theorem 4.3: The maximization of submodular function in Eqn. 33 using greedy algorithm guarantees $(1 - \frac{1}{e})$ approximation

Proof: For proof, refer to appendix D. ■

The greedy algorithm is presented in Algorithm 1. In the first step, $\tilde{\mathcal{N}}_p$ is initialized for all $p = 1, 2, \dots, \mathcal{V}'$. $\tilde{\mathcal{N}}$ is defined in the line 2. For each iteration, a new element with the highest marginal gain from the set is added to the set X , and consequently removed from the ground set Y in lines 5-7. The process continues until the size of the set X exceeds the number of servers or the marginal gain obtained by adding the element is not equal to zero.

However, as the number of tasks and vehicles increase in a time slot, it is challenging to implement the greedy approach for large-scale scheduling problems and to handle large-scale networks, an effective task scheduling algorithm is required.

Algorithm 1 Greedy Task Scheduling Algorithm

Input: Set of tasks $\mathcal{T}$, set of Vehicles $\mathcal{V}$, sojourn time Δ for all vehicles.

Output: X : Scheduling set

```

1: Initialize  $\tilde{\mathcal{N}}_p = \phi, n = 1, 2, \dots, \mathcal{V}'$ 
2:  $\tilde{\mathcal{N}} = \bigcup_{p=1}^{\mathcal{V}'} \tilde{\mathcal{N}}_p$ 
3:  $X = \tilde{\mathcal{N}}, Y = \mathcal{N}$ 
4: repeat
5:   Choose an element  $\hat{e} = x_{v,i}^r$ , i.e.,  $\hat{e} = \operatorname{argmax}_{e' \in Y} U(X|e')$ 
6:    $X = X \cup \{\hat{e}\}$ 
7:    $Y = Y - \{\hat{e}\}$ 
8: until  $|X| \leq \mathcal{R}^{max}$  or  $U(X|e') \neq 0$ 
9: return  $X$ 

```

Hence, we propose a hybrid optimization task scheduling approach presented in the following section.

V. SA-PSO TASK SCHEDULING ALGORITHM

There exist many studies to prove that metaheuristic optimization algorithms optimally solve NP-hard problems due to their fast convergence, low complexity of implementation, and high robustness [16, 34]. They have been applied widely in complex real-time problems. However, metaheuristic algorithms have their respective disadvantages along with advantages. The particle swarm optimization (PSO) technique suffers from premature convergence, which leads to the optimal local trap with highly complex search spaces [35]. On the other hand, simulated annealing (SA) applies the metropolis acceptance rule that helps to move the solution to the optimal global solution. Hence, it can avoid the optimal local trap. However, it has a slow convergence [36]. Therefore, this work extracts and incorporates the features of the PSO and SA algorithms and proposes a hybrid optimization algorithm for the task scheduling problem.

A. Particle Swarm Optimization

PSO algorithm contains the population that updates the particles' position and velocities by learning the particles' experiences [35]. Let $\mathbb{S}$ be the size of the swarm in PSO and x_i and v_i be the position and the speed of the particle i ($i = 1, 2, \dots, |\mathbb{S}|$). Each position of the particle includes the $V * n$ elements, which represent the number of tasks of vehicles in a particular time instant. Elements' positions are arranged in vehicular order. Let's take an example of the task set with five tasks. Consider one of the swarm particles as $[5, 4, 0, 1, 2]$, which indicates that the first task is assigned to the fifth RSU server, the second task is assigned to the fourth RSU server, and so on. Where 0 indicates that the task is allocated vehicle. The particle's velocity represents the span of the offloading tasks to other RSU servers. For example, the velocity vector is $[1, 0, 1, 2, 3]$, which indicates that, in the iterative process, the first task is moved to the RSU server with id 6, i.e., $(5+1)\%m$, where m is the number of servers. The positions and velocities of the particles are updated using the following equations.

$$v_{i+1} = \alpha v_i + c_1 r_1 (x_i^l - x_i) + c_2 r_2 (x_i^g - x_i) \tag{35}$$

$$x_{i+1} = x_i + v_{i+1} \tag{36}$$

Algorithm 2 SA-PSO Task Scheduling Algorithm

Input: Set of tasks $\mathcal{T}$, set of vehicles $\mathcal{V}$, sojourn time Δ for all vehicles

Output: X : Scheduling set

```

1: Initialize the PSO parameters  $c_1, c_2, r_1, r_2$ , and inertia  $\alpha$ 
2: Initialize the temperature  $t$ , and cooling rate  $temp$  of SA
3: Initialize the number of iterations  $max_{itr}$ 
4: for each  $i = 1$  to  $S$  do
5:   Initialize the particle position  $x_i$ 
6:   Initialize the particle velocity  $v_i$ 
7: end for
8: while  $itr \leq max_{itr}$  do
9:   for each  $i = 1$  to  $S$  do
10:    Calculate fitness value  $F_i$  of the  $i^{th}$  particle
        using Eqn. 21
11:   end for
12:   Update the global position  $x^g$  of the population
13:   for each  $i = 1$  to  $S$  do
14:    Update the velocity of the particle  $v_{i+1}$  using Eqn. 35
15:    Update the position of the particle  $x_i$  using Eqn. 36
16:    if  $F_{i+1} \geq F_i$  then
17:      Accept the new update  $x_{i+1}$ 
18:    else
19:      SA( $x_{i+1}, v_{i+1}, F_{i+1}, t$ ) using Algorithm 3.
20:    end if
21:   end for
22:   Update the annealing temperature  $t \leftarrow temp * t$ 
23:    $iter \leftarrow iter + 1$ 
24: end while
25:  $X \leftarrow x^g$ 
26: return  $X$ 

```

Where, c_1 and c_2 are the constants, r_1 and r_2 are the random numbers. x_i^l represents the local best of the particle and x_i^g denotes the global best of the population.

B. Simulated Annealing

The simulated annealing algorithm provides a solution to escape from the local optimum trap by observing the states which worsen the fitness value, referred to as a jumping mechanism [36]. In the iterative process, a new solution is generated, and the difference in the cost function is computed. The new state is accepted if the difference is greater than 0. Otherwise, it is accepted with the probability $e^{-diff/t}$, where $diff$ indicates the difference between the new solution and previous solution, and t is the annealing temperature. Since SA has the advantage of escaping from the local optimal, we combined PSO with SA for the efficient convergence and exploration of our hybrid task scheduling algorithm.

Algorithm 3 Simulated Annealing (SA) Algorithm

Input: Particle Updated position x_i and velocity v_i , Temperature t

```

1:  $diff = F_{i+1} - F_i$ 
2: Generate the random number  $r$  between 0 and 1
3: if  $r < e^{-diff/t}$  then
4:   Accept the new particle's position  $x_{i+1}$  and velocity  $v_{i+1}$ 
5: else
6:   Keep the previous particle's position  $x_i$  and velocity  $v_i$ .
7: end if

```

The proposed hybrid optimization algorithm to solve the mobility-aware task scheduling problem is shown in Algorithm

TABLE II: Simulation Parameters

Parameters	Values
Number of RSUs	10-50
Number of vehicles	20-500
Mobility model	MRP
Vehicle CPU frequency	2-5 GHz
Fog server CPU frequency	10-15 GHz
Average task data size	0.5-5 MB
Required task computing size (10^9 cycles)	0.5-5
Transmission power of RSU server	50 dBm
RSU server bandwidth	50 MHz
Allowable task deadline	50-200 ms
SA-PSO Parameters	Values
Poulation size	50
inertia(α)	0.6-1.0
c_1	2.1
c_2	2.1
r_1	2.1-2.5
r_2	2.1-2.3
$temp$	0.8
Iterations	1000

2. The algorithm takes the set of tasks $\mathcal{T}$ and RSUs $\mathcal{R}$ as input and gives the scheduling set X as output. At the beginning of the algorithm, PSO parameters, such as the size of the swarm, particle positions, and velocities, and SA parameters, such as temperature and cooling parameters, are initialized in lines 1-7. For the given number of iterations in line 8, the algorithm calculates the fitness of each particle i in the swarm and updates the global best of the particles x_i^g in lines 9-12. For each particle i , the velocity and position are updated in lines 14-15. If the fitness of the newly updated particles is greater than the previous article, then the algorithm keeps the updated particle's position, as shown in lines 16-17. Otherwise, it applies the SA technique using the Algorithm. 3, which uses the metropolis acceptance rule to update or keep the particle's position and velocity as shown in lines 2-6 of Algorithm 3. The loop in Algorithm 2 continues until it reaches the termination condition. At the end of each iteration, the temperature is updated. After the loop terminates, the global best of particles, i.e., the scheduling set, is returned as output.

VI. PERFORMANCE EVALUATION

In this section, the efficacy of the proposed mobility aware task scheduling algorithm is discussed. Simulation settings, performance metrics and benchmarks for comparison are presented.

A. Simulation Settings

We evaluate the performance of the proposed **SA-PSO algorithm using python in jupyter notebook**. Table II shows the simulation parameters and their values. We consider a vehicular fog network that consists of 3-15 RSUs with one base station, and vehicles with onboard processors are considered in the range of 10-50. In the vehicular fog computing (VFC) network, all RSUs are randomly deployed, and each RSU is connected to BS. **The simulation parameter values are referred from [31, 37].** The transmission power of each RSU

is surmised to be 50 dBm. The bandwidth of RSU servers is assumed to be in the range of 50 MHz. The RSU server CPU frequency is considered in the 10-15 GHz range. The vehicle onboard processor CPU frequency is assumed to be in the range of 2-5 GHz. The average size of the task is considered in the range of 0.5 to 5 MB. The proposed SA-PSO algorithm parameters are also listed in Table II. The population size is taken as 50. The PSO parameters such as c_1 , c_2 , r_1 , and r_2 are considered according to [38], and tested with 2.1, 2.1, 2.5 and 2.5 respectively, the inertia of particle is taken in the range of 0.6 to 1.0. And the SA parameter, cooling rate $temp$ is set in the range 0.96 to 1.0, with the initial temperature 1000 [36]. The algorithm runs for 1000 iterations. **For reliability purpose, the authors have evaluated the proposed mechanisms using the publicly accessible code of the papers [29, 39] by adopting them according to our proposed network model.** Experiments are conducted 50 times for the simulation environment, and the results are drawn for an average of 50 experiments. We have compared our proposed algorithm with existing state-of-the-art algorithms in the considered simulation environment.

B. Benchmark Algorithms

The proposed SA-PSO task scheduling algorithm is compared with six benchmark schemes, i.e., MACTER [6], MATO [10], RADQN [25], ORADRL [26], random scheduling and RSU scheduling, illustrated below.

1) **MACTER [6]**: The mobility-aware computational efficiency-based task offloading and resource allocation (MACTER) scheme offloads vehicular tasks and allocates computation resources using game theory in a VFC network. Tasks are scheduled to either locally or at RSU. But it does not consider the future mobility information of vehicles.

2) **MATO [10]**: Mobility aware task offloading strategy offloads the tasks and allocates them in a vehicular fog server using matching theory. Tasks are offloaded either to vehicles or to RSUs. It considers the mobility of the vehicle only in one RSU.

3) **RADQN [25]**: Resource allocation using a deep Q network (DQN) in which edge resources are allocated to the vehicular tasks using the DQN mechanism without considering mobility.

4) **ORADRL [26]**: Offloading and resource allocation using a deep reinforcement learning (DRL) approach in which tasks are allocated to the server using the DRL technique without considering mobility.

5) **Random Scheduling**: Random scheduling schedules the vehicular tasks of a vehicle to the randomly to RSU server or vehicle.

6) **RSU Scheduling**: In the RSU scheduling strategy, the tasks are offloaded uniformly to the RSU servers that are available in the vehicle path rather than offloading to vehicles.

C. Performance Metrics

The proposed approach is compared with benchmark schemes against the below performance metrics.

1) **Service Delay**: It is the latency of the total time of the vehicle's tasks calculated using Eqn. 17. The service latency involves the computation time concerning the vehicle if it's executed locally. It involves communication latency, and computation time if it is offloaded to the RSU server.

2) **Energy Consumption**: The energy consumed in the vehicle to process all its vehicular tasks is computed using Eqn. 18.

3) **Quality of service**: Quality of service is measured by computing the service delay of the tasks and energy consumption of the vehicles, which is our objective function indicated in Eqn. 21. It helps to trade off the vehicular tasks' service time and energy consumption of the vehicles. This metric provides the vehicular user's satisfaction achieved by the proposed algorithm.

4) **Success Rate**: Success rate is the percentage of the successful tasks to the number of tasks in the base station. Where successful tasks are the tasks that are completed before the deadline of the tasks.

D. Mobility Model

We employ a real-time data set released by Crawdad [40] to assess the performance of the proposed mobility-aware task scheduling method. The data set covers vehicle mobility traces in San Francisco. It comprises the coordinates of 500 vehicles tracked over the course of thirty days in the San Francisco Bay Area. In our scenario, we take into account 50 vehicles with 15 RSUs that are densely deployed close to the roads, and the RSU locations are recorded using (x, y) coordinates. In our simulation, the base station predicts the sojourn time of the vehicle at each RSU and allocates the tasks to the RSUs and vehicles based on the vehicles' mobility information.

E. Results and Discussion

Experiment results with the proposed algorithm are provided below with all existing standard algorithms. We observed the performance of the system in three different scenarios, i.e., (i) impact of change in tasks, (ii) impact of change in vehicles and (iii) impact of change in RSUs.

1) **Impact of Number of Tasks**: In this subsection, we depict the efficacy of the proposed SA-PSO task scheduling technique by changing the number of tasks from 50 to 250 by considering the number RSUs as fixed to 10. is as shown in Fig. 2. From Fig. 2a, it can be noticed that the service delay of the tasks increases while increasing the tasks due to the burden on the RSU servers. It can be seen that the service delay of tasks for the proposed SA-PSO is low compared to the other algorithms. By distributing vehicular tasks to servers according to vehicle mobility and sojourn time, the proposed algorithm can effectively reduce service time. The proposed SA-PSO has an improvement of up to 61.8%, 56%, 48%, 41%, 33%, and 19.2% compared to the existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER, respectively. The overall energy consumption of the vehicles is increasing while increasing the tasks in our network, as shown in Fig. 2b. The energy consumption of the vehicles is less for the proposed algorithm since our objective includes energy consumption minimization. Moreover, the proposed

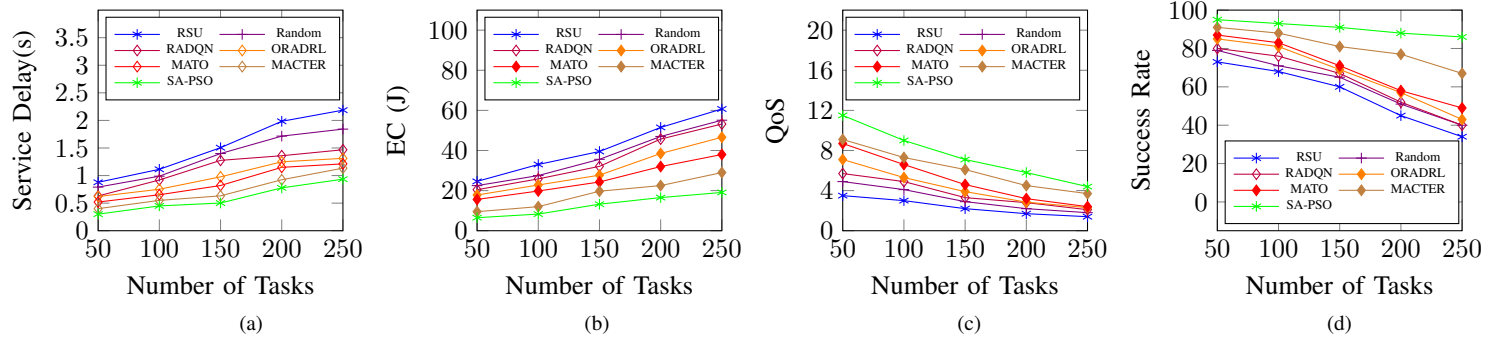


Fig. 2: Proposed SA-PSO and existing algorithms performance with Number of Tasks vs (a) Service Delay (b) Energy Consumption (c) Quality of Service (d) Success Rate

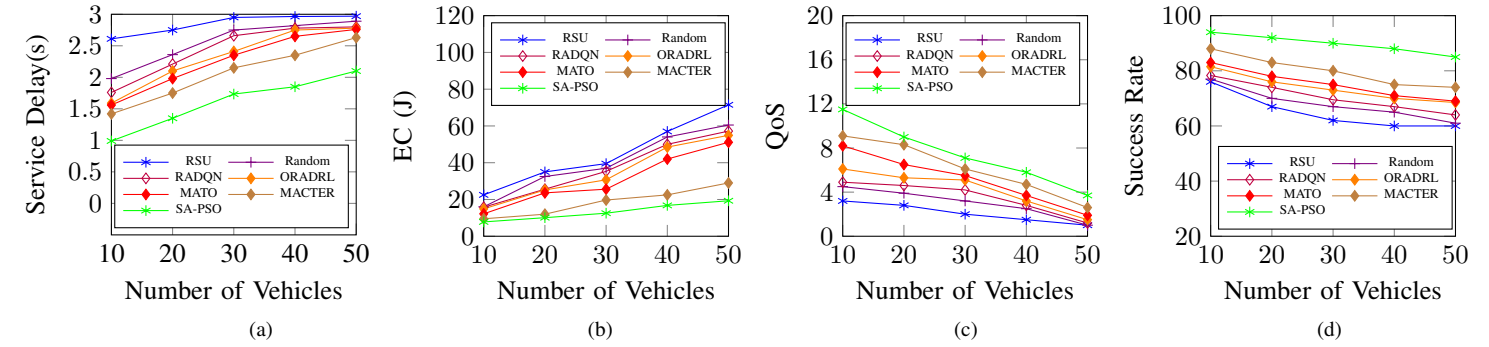


Fig. 3: Proposed SA-PSO and existing algorithms performance with Number of Vehicles vs (a) Service Delay (b) Energy Consumption (c) Quality of Service (d) Success Rate

SA-PSO has an improvement of up to 69.6%, 66.4%, 62.8%, 57%, 51.6%, and 16% compared to the existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER respectively.

We evaluate the proposed algorithm with the quality of the service metric shown in Fig. 2c. The QoS decreases while the number of tasks increases in the VFC network. It can be depicted that the QoS for the proposed algorithm is high when compared to the other algorithms since its objective function, i.e., utility function, includes the minimization of service time and energy consumption of the vehicles. From Fig. 2d, it is depicted that the success rate of the tasks is decreasing while increasing the number of tasks. It can be observed that the success rate for the proposed algorithm is more when compared to the other algorithms. Since the objective of our proposed algorithm includes the service time of the tasks in the utility function, it is possible to get a high success rate with the proposed algorithm.

2) Impact of Number of Vehicles: In this subsection, we depict the performance of the proposed SA-PSO task scheduling algorithm by changing the number of vehicles from 10 to 50, by considering the number of RSUs are fixed to 10 as shown in Fig. 3. From Fig. 3a, it can be noticed that increasing the number of vehicles leads to an increase in the service latency of the tasks. As we discussed earlier, vehicles have a different number of tasks. When the vehicles

are increased in a path, RSU servers must execute the tasks with their capacity. Hence the service time is more with more number of vehicles. It can be observed that the service time of tasks for the proposed SA-PSO is low compared to the other algorithms. As the tasks are distributed to RSU based on vehicle mobility and sojourn time, the service time for the tasks in the proposed algorithm is very less. The proposed SA-PSO has an improvement of up to 44.2%, 38.4%, 34.4%, 30.8%, 29.2%, and 16% compared to the existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER respectively. The overall energy consumption of the vehicles is increasing while increasing the vehicles in our network, as shown in Fig. 3b. The energy consumption of the vehicles is less for the proposed algorithm since our objective includes energy consumption minimization, and it allocates the tasks by considering the available energy of the vehicle. Moreover, the proposed SA-PSO has an improvement of up to 69.4%, 64.2%, 61.2%, 58.8%, 53.2%, and 14% compared to existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER respectively.

The proposed algorithm is evaluated with the quality of the service metric is shown in Fig. 3c. The QoS is low, with more vehicles in the VFC network. It can be depicted that the proposed algorithm achieves high quality of service compared to other algorithms. The proposed mobility-aware task scheduling algorithm prioritizes the reduction of service

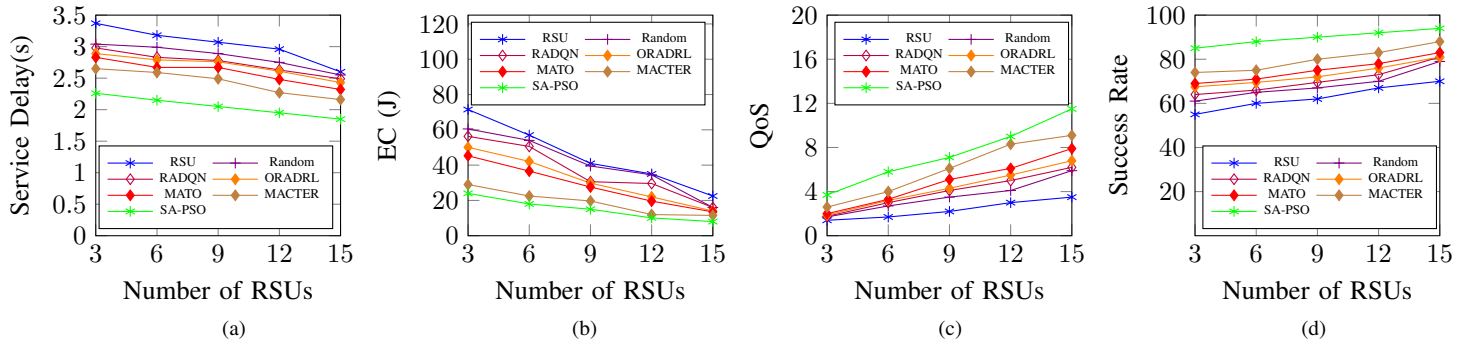


Fig. 4: Proposed SA-PSO and existing algorithms performance with Number of RSUs vs. (a) Service Delay (b) Energy Consumption (c) Quality of Service (d) Success Rate

latency and energy consumption of vehicles by taking into account the vehicle's mobility. Hence, it can enhance the QoS efficiently. From Fig. 3d, it is observed that the success rate of the tasks is low with more number of vehicles. It can be observed that the success rate for the proposed algorithm with a change in the number of vehicles is more when compared to the other task scheduling mechanisms. By incorporating the service delay in the utility function, the algorithm allocates the vehicular tasks according to its delay constraint. Hence, the proposed SA-PSO achieves a notably high success rate.

3) *Impact of Number of RSUs:* In this subsection, we have depicted the performance of the proposed SA-PSO task scheduling algorithm by changing the number of the RSUs from 3 to 15, by fixing the number of vehicles to 30 and the number of tasks to 150 as shown in Fig. 4. From Fig. 4a, it can be noticed that the service time of the tasks decreasing while increasing the number of RSUs. It can be observed that the service time of tasks for the proposed SA-PSO is low compared to the other algorithms. The proposed algorithm efficiently minimizes the service time for tasks by allocating to RSUs based on the vehicle's mobility and sojourn time. The proposed SA-PSO has an improvement of up to 32.2%, 28.4%, 24.8%, 23.2%, 20.6% and 12% compared to the existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER, respectively. The overall energy consumption of the vehicles is decreasing while increasing the number of RSUs in our network, as shown in Fig. 4b. The energy consumption of the vehicles is less for the proposed algorithm since our objective aims to minimize energy consumption. Moreover, the proposed SA-PSO has improved up to 66.4%, 61.6%, 57%, 50.4%, 41.4%, and 16% compared to the existing algorithms RSU, Random, RADQN, ORADRL, MATO and MACTER, respectively.

We evaluate the proposed algorithm with the quality of the service metric shown in Fig. 4c. The QoS increases while the number of RSUs increases in the VFC network. It can be depicted that the QoS for the proposed algorithm is high when compared to the other algorithms since its objective function, i.e., utility function, includes the minimization of service time and energy consumption of the vehicles. From Fig. 4d, it is depicted that the success rate of the tasks increases

while increasing the number of RSUs. It can be observed that the success rate for the proposed algorithm is more when compared to the other algorithms. The proposed SA-PSO can yield a significantly high success rate since the algorithm allocates the tasks to minimize the service time constrained to the deadline.

The proposed SA-PSO algorithm introduces a novel task scheduling mechanism by considering factors such as the vehicle's mobility, available resources, and task deadlines. In contrast, the existing approaches like MATO and MACTER limit task allocation to the available resources within the corresponding RSU region. However, if a vehicle moves out of the RSU region after the task allocation, the allocated task fails according to the existing algorithms due to a lack of continuity. The approaches such as RADQN and ORADRL constrain the task allocation in one RSU region without considering mobility, which leads to the failure of the tasks' execution, resulting in higher delay and energy consumption.

The proposed SA-PSO algorithm overcomes the limitation by considering the vehicle's mobility and available resources beyond the corresponding RSU region. As a result, it reduces the service time of the vehicular tasks and energy consumption of the vehicle compared to the existing algorithm. Hence, The proposed algorithm optimizes the task allocation and improves the overall performance of the network.

F. Sensitivity Analysis

Sensitivity analysis determines the impact of changes in input parameters on the results of an algorithm. It helps in understanding how sensitive the output is to various input parameters and enables identifying critical factors which affect the performance of the algorithm [41].

In this section, we present the sensitivity analysis of the proposed SA-PSO in terms of PSO and SA parameters such as c_1 , c_2 , α , and $temp$, respectively, as shown in Table. III. It is observed that the fitness value of the proposed SA-PSO algorithm is decreasing while increasing the trust parameter, cognitive constant c_1 from 2.1 to 2.5. Increasing the c_1 value leads to the exploitation of the solution in the local search space. Hence the fitness value is lower with the higher c_1 value. The proposed SA-PSO's fitness value is higher with

TABLE III: Sensitivity Analysis

Parameter	Value	Fitness	Parameter	Value	Fitness
c_1	2.1	12.16	Inertia (α)	0.6	12.26
	2.2	12.06		0.7	12.15
	2.3	11.75		0.8	12.05
	2.4	11.25		0.9	11.95
	2.5	11.1		1.0	11.85
c_2	2.1	12	$temp$	0.6	11.99
	2.2	11.7		0.7	12.1
	2.3	11.2		0.8	12.24
	2.4	11.05		0.9	11.87
	2.5	10.9		1.0	11.64

a higher value of the social constant c_2 . In the proposed hybrid SA-PSO algorithm, when the cognitive constant c_1 is much larger, the particle tends to exhibit a stronger attraction towards its personal best position resulting in excessive focus on individual performance. Similarly, when social constant c_2 is larger, particles are drawn towards the global best position, which results in premature convergence towards sub-optimal solutions. Therefore, to balance the individual best positions and collective best positions, the values of cognitive and social constants are considered 2.1, and 2.1, respectively.

The inertia weight governs the impact of a particle's previous momentum when updating its velocity and enables control over its trajectory resulting in a trade-off between exploration and exploitation. The proposed hybrid SA-PSO is evaluated by changing the inertia α from 0.6 to 1.0. It can be noticed that the larger values of inertia weight exhibit inefficient performance. Moreover, we have evaluated the performance of the proposed SA-PSO by changing the parameter, $temp$ value from 0.6 to 1.0. From Table. III, it shows that the proposed SA-PSO with a cooling rate of 0.8 achieves the balance between exploration and exploitation. The gradual increase of $temp$ from 0.6 to 0.8 enables adequate exploration in the initial stages which helps to escape local optima. Furthermore, as the cooling rate increases from 0.8 to 1.0, the algorithm is inclined towards more exploitation with a faster cooling rate.

G. Convergence of Proposed SA-PSO

The convergence of the proposed SA-PSO algorithm is depicted in Fig. 5. The proposed SA-PSO is compared with

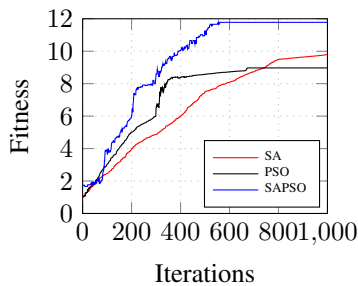


Fig. 5: Convergence comparison of proposed SA-PSO with SA and PSO

the simulated annealing algorithm and the particle swarm optimization algorithm based on the fitness value of the mobility

ware task scheduling problem presented in Eqn. 21. From the figure, it can be noticed that the convergence of particle swarm optimization is fast and restricted to lower fitness value. But the simulated annealing algorithm's convergence gradually increases to the higher fitness value. Combining both the SA and PSO techniques, the proposed SA-PSO has the highest convergence value compared to both the SA and PSO.

H. Limitations

The proposed work presents a mobility-aware task-scheduling strategy in vehicular fog networks, which allocates the vehicular tasks to the resources efficiently by considering the mobility of the vehicles and available resources using a hybrid SA-PSO optimization technique. However, various constraints and limitations require attention for practical execution. First, in real world scenarios, vehicular tasks and RSU servers often exhibit diversity. For instance, certain tasks (for example, high definition video streaming, virtual reality, etc.) may require a graphical processing unit (GPU), i.e., GPU-enabled fog servers. Hence, modifying the proposed algorithm is essential for practical use in such heterogeneous environments. **Second, in this study, we assumed that each VFC server can execute only one task at a time.** However, simultaneous execution of multiple tasks within a single fog server can be considered.

Third, the communication latency in vehicular fog networks is influenced by several obstacles, such as buildings, terrain, and other structures that can obstruct wireless signals and introduce signal attenuation, which is not considered as a constraint in the proposed work. Fourth, the underutilization of computation resources available in vehicles. The proposed work considers scheduling the tasks to either vehicles onboard CPU or RSU servers. **However, the potential of computation capabilities of the neighbour vehicles has been underexplored. Vehicular fog networks inherently possess diverse computing resources that can significantly enhance the network's performance. These constraints and limitations may be considered for the practical execution of the proposed solution.**

VII. CONCLUSION AND FUTURE WORK

This paper presents a mobility-aware intelligent task scheduling mechanism for fog enabled IoV networks. Initially, a task scheduling problem is formulated into mixed integer non-linear programming (MINLP) to minimize the service delay of the vehicular tasks and energy consumption of the vehicles. Then, the Markov renewal process (MRP) is used to determine the vehicle's mobility and sojourn time. The task scheduling problem is solved using the greedy algorithm with a 0.63 approximation for small-scale networks. Further, a simulated annealing based particle swarm optimization based task scheduling mechanism is designed for large-scale networks. The proposed mechanism is evaluated by comparing it with the benchmark algorithms. The proposed algorithm performs efficiently and has an improvement of 15% in minimization of the service delay of the tasks and has an improvement of 16% in minimization energy consumption of the vehicles compared to the second-best algorithm. **In future work, we will incorporate the cooperation between the RSUs in the resource**

allocation process to improve the performance of vehicular fog networks.

APPENDIX

A. NP Hardness Proof

Consider a set of vehicular tasks $\mathcal{T} = \{t_1, t_2, \dots, t_i, \dots, t_n\}$, and a set of fog nodes including RSU servers and vehicular processor $\mathcal{F} = \{f_1, f_2, \dots, f_j, \dots, f_m\}$ with heterogeneous processing capacities. Follow the conditions.

- Each task t_i can only be processed by one server f_j
- A limit is set on the maximum number of tasks allocated to a fog node.

If a problem can be reduced to NP class, it is considered NP-hard. In this proof, we reduced the bin-packing problem, a known NP-hard problem, to our objective function. Let us consider a collection of items, denoted as $I = \{l_1, l_2, \dots, l_i, \dots, l_n\}$, where each item l_i has a value that belongs to $\{0,1\}$. And a set of bins $B = \{b_1, b_2, \dots, b_j, \dots, b_m\}$ of unit capacity. In the bin-packing problem, the objective is to assign each item l_i to a distinct bin b_j , and the allocation is represented as d_{ij} . The sum of the number in each $b_j \in B$ should not exceed one, i.e., $\sum_{i \in I} d_{ij} \leq 1$, such that the total number of used bins is minimized. Relating to our problem, items are incoming tasks T and, bins are available servers F . And the scheduling of the vehicular tasks to servers can be written as $U(t_i, f_j)$, which is similar to the bin-packing assignment problem d_{ij} , i.e., $U(t_i, f_j) = d_{ij}$.

Thus, the NP-hardness of our objective function is proved.

B. Proof of Lemma 4.1

Matroid $\mathcal{M}$ satisfies the three conditions. According to normalization and hereditary properties of the sets, the first and second conditions hold true. To prove the third condition is true, assume $\mathcal{I}_1, \mathcal{I}_2 \in \mathcal{I}$, and $|\mathcal{I}_1| < |\mathcal{I}_2|$, there exists an element $p \in \{1, 2, \dots, \mathcal{N}\}$ which satisfies the condition $|\mathcal{I}_1 \cap \mathcal{N}_p| < |\mathcal{I}_2 \cap \mathcal{N}_p|$. Without loss of generality, assume there exists one more index with $n_{\bar{p}}$ which also satisfies the condition $|\mathcal{I}_1 \cap \mathcal{N}_{\bar{p}}| < |\mathcal{I}_2 \cap \mathcal{N}_{\bar{p}}|$. Consider an element $e = x_{v,i}^r \in \mathcal{N}_{\bar{p}} \cap (\mathcal{I}_1 \setminus \mathcal{I}_2)$ which satisfies all the constraints in the problem formulation related to Eqn. 21, is added to the set $\mathcal{I}_1$, and the new obtained set is still belong to $\mathcal{I}$, i.e., $\mathcal{I}_1 \cup \{e\}$. Hence, the third condition for the matroid $\mathcal{M} = (\mathcal{N}, \mathcal{I})$ has been proved.

C. Proof of theorem 4.2

The objective function U presented in Eqn. 21 is a monotone submodular function. We define the $X \subseteq Y \in \mathcal{I}$ and let $e \in \mathcal{N} \setminus Y$. Benefit of adding $e = x_{v,i}^r$ to X is:

$$U(X \cup \{e\}) = U(X) + U(\hat{e}) \quad (37)$$

To this end, a new element $\hat{e} = x_{v,i}^r \in \mathcal{N}$ is adding to the set X , then the marginal gain obtained by adding new element $U(X|\hat{e})$ is given below:

$$\left(\sum_{x_{v',i'}^r \in X} \frac{1}{x_{v',i'}^r} + \frac{1}{w + x_{v,i}^r} \right) - \left(\sum_{x_{v',i'}^r \in X} \frac{1}{x_{v',i'}^r} \right) = \frac{1}{w + x_{v,i}^r} \quad (38)$$

Where w refers to the gradient of waiting time obtained due to adding $x_{v,i}^r$ in the set X and from the above equation, the marginal gain $U(X|\hat{e})$ is greater than zero. Same new element $\hat{e} = x_{v,i}^r \in \mathcal{N}$ is adding to the set Y , then the marginal gain obtained by adding new element $U(Y|\hat{e})$ is given below:

$$\left(\sum_{x_{v',i'}^r \in Y} \frac{1}{x_{v',i'}^r} + \frac{1}{w' + x_{v,i}^r} \right) - \left(\sum_{x_{v',i'}^r \in Y} \frac{1}{x_{v',i'}^r} \right) = \frac{1}{w' + x_{v,i}^r} \quad (39)$$

Where w' refers to the gradient of waiting time obtained by adding $x_{v,i}^r$ in set Y and $w' \geq w$, since the task's waiting time is more in the bigger set Y than, the smaller set X . From the above equation, the marginal gain $U(Y|\hat{e})$ is greater than zero. Therefore, the marginal gain differences such as $U(X \cup \{e\}) - U(X) \geq U(Y \cup \{e\}) - U(Y)$ is greater than or equal to zero. Hence the problem U is proved as a monotone submodular function.

D. Proof of theorem 4.3

To prove that the greedy algorithm guarantees 0.63 approximation for maximization of submodular function in Eqn. 33. Assume:

$X_i = \{x_{v,1}^r, x_{v,1}^r, \dots, x_{v,i}^r\}$ is the scheduling set provided by greedy algorithm.

$X^* = \{x_{v,1}^{r*}, x_{v,2}^{r*}, \dots, x_{v,i}^{r*}\}$ is the optimal scheduling set.

$\mathcal{OPT} = U(X^*)$ as the optimal scheduling set.

To prove $U(X_i) \leq (1 - \frac{1}{e})\mathcal{OPT}$

$$U(X^*) \leq U(X_i \cup X^*), \because \text{Monotonicity} \quad (40)$$

$$= U(X_i) + \sum_{i \in X^*} U(x_{v,i}^{r*} | X_i \cup \{x_{v,1}^{r*}, x_{v,2}^{r*}, \dots, x_{v,i-1}^{r*}\}) \quad (41)$$

$$= U(X_i) + \kappa U(i+1 | X_i) \because \text{Submodularity} \quad (42)$$

We can rewrite the Eqn. 42 as:

$$U(i+1 | X_i) \geq \frac{1}{\kappa} (\mathcal{OPT} - U(X_i)) \quad (43)$$

Let us assume $\varrho_i = \mathcal{OPT} - U(X_i)$ which infers that

$$U(X_{i+1}) - U(X_i) = \varrho_i - \varrho_{i+1} = U(i+1 | X_i). \quad (44)$$

Therefore, Eqn. 43 can be written as:

$$\varrho_i - \varrho_{i+1} \geq \frac{1}{\kappa} \varrho_i \quad (45)$$

$$\varrho_{i+1} \leq \left(1 - \frac{1}{\kappa}\right) \varrho_i \quad (46)$$

$$\text{Hence, the recursive eqn is } \varrho_{\kappa} = \left(1 - \frac{1}{\kappa}\right)^{\kappa} \varrho_0 \quad (47)$$

As we know, $1 - x \leq e^{-x}, \forall x \in \mathbb{R}$, then:

$$\varrho_{\kappa} \leq \left(1 - \frac{1}{\kappa}\right)^{\kappa} \mathcal{OPT} \leq \frac{1}{e} \mathcal{OPT} \quad (48)$$

$$\mathcal{OPT} - U(X_k) \leq \frac{1}{e} \mathcal{OPT} \quad (49)$$

$$U(X_k) \geq \left(1 - \frac{1}{e}\right) \mathcal{OPT} \quad (50)$$

Thus, the theorem was proved.

REFERENCES

- [1] J. Liu, M. Ahmed, M. A. Mirza, W. U. Khan, D. Xu, J. Li, A. Aziz, and Z. Han, "RI/drl meets vehicular task offloading using edge and vehicular cloudlet: A survey," *IEEE Internet of Things Journal*, vol. 9, no. 11, pp. 8315–8338, 2022.
- [2] Y. Liu, H. Zhang, K. Long, H. Zhou, and V. C. Leung, "Fog computing vehicular network resource management based on chemical reaction optimization," *IEEE Transactions on Vehicular Technology*, vol. 70, no. 2, pp. 1770–1781, 2021.
- [3] P. Arthurs, L. Gillam, P. Krause, N. Wang, K. Halder, and A. Mouzakitis, "A taxonomy and survey of edge cloud computing for intelligent transportation systems and connected vehicles," *IEEE Transactions on Intelligent Transportation Systems*, 2021.
- [4] H. Wang, T. Liu, B. Kim, C.-W. Lin, S. Shiraishi, J. Xie, and Z. Han, "Architectural design alternatives based on cloud/edge/fog computing for connected vehicles," *IEEE Communications Surveys & Tutorials*, vol. 22, no. 4, pp. 2349–2377, 2020.
- [5] I. Martinez, A. S. Hafid, and A. Jarray, "Design, resource management, and evaluation of fog computing systems: a survey," *IEEE Internet of Things Journal*, vol. 8, no. 4, pp. 2494–2516, 2020.
- [6] S. Raza, S. Wang, M. Ahmed, M. R. Anwar, M. A. Mirza, and W. U. Khan, "Task offloading and resource allocation for iov using 5g nr-v2x communication," *IEEE Internet of Things Journal*, 2021.
- [7] S. Misra and S. Bera, "Soft-van: Mobility-aware task offloading in software-defined vehicular network," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 2, pp. 2071–2078, 2019.
- [8] A. M. A. Hamdi, F. K. Hussain, and O. K. Hussain, "Task offloading in vehicular fog computing: State-of-the-art and open issues," *Future Generation Computer Systems*, 2022.
- [9] K. Zhang, M. Peng, and Y. Sun, "Delay-optimized resource allocation in fog-based vehicular networks," *IEEE Internet of Things Journal*, vol. 8, no. 3, pp. 1347–1357, 2020.
- [10] P. Liu, J. Li, and Z. Sun, "Matching-based task offloading for vehicular edge computing," *IEEE Access*, vol. 7, pp. 27 628–27 640, 2019.
- [11] J. Sun, Q. Gu, T. Zheng, P. Dong, A. Valera, and Y. Qin, "Joint optimization of computation offloading and task scheduling in vehicular edge computing networks," *IEEE Access*, vol. 8, pp. 10 466–10 477, 2020.
- [12] Y. Wang, P. Lang, D. Tian, J. Zhou, X. Duan, Y. Cao, and D. Zhao, "A game-based computation offloading method in vehicular multiaccess edge computing networks," *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 4987–4996, 2020.
- [13] Z. Zhou, P. Liu, J. Feng, Y. Zhang, S. Mumtaz, and J. Rodriguez, "Computation resource allocation and task assignment optimization in vehicular fog computing: A contract-matching approach," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 4, pp. 3113–3125, 2019.
- [14] P. Qin, Y. Fu, G. Tang, X. Zhao, and S. Geng, "Learning based energy efficient task offloading for vehicular collaborative edge computing," *IEEE Transactions on Vehicular Technology*, 2022.
- [15] Z. Zhou, J. Feng, Z. Chang, and X. Shen, "Energy-efficient edge computing service provisioning for vehicular networks: A consensus admm approach," *IEEE Transactions on Vehicular Technology*, vol. 68, no. 5, pp. 5087–5099, 2019.
- [16] M. Abdel-Basset, R. Mohamed, M. Elhoseny, A. K. Bashir, A. Jolfaei, and N. Kumar, "Energy-aware marine predators algorithm for task scheduling in iot-based fog computing applications," *IEEE Transactions on Industrial Informatics*, vol. 17, no. 7, pp. 5068–5076, 2020.
- [17] Y. Zhai, W. Sun, J. Wu, L. Zhu, J. Shen, X. Du, and M. Guizani, "An energy aware offloading scheme for interdependent applications in software-defined iov with fog computing architecture," *IEEE Transactions on Intelligent Transportation Systems*, vol. 22, no. 6, pp. 3813–3823, 2020.
- [18] T. Liu, J. Li, F. Shu, and Z. Han, "Optimal task allocation in vehicular fog networks requiring urlhc: An energy-aware perspective," *IEEE Transactions on Network Science and Engineering*, vol. 7, no. 3, pp. 1879–1890, 2019.
- [19] A. Hazra, M. Adhikari, T. Amgoth, and S. N. Srirama, "Fog computing for energy-efficient data offloading of iot applications in industrial sensor networks," *IEEE Sensors Journal*, vol. 22, no. 9, pp. 8663–8671, 2022.
- [20] Q. Luo, C. Li, T. H. Luan, and W. Shi, "Minimizing the delay and cost of computation offloading for vehicular edge computing," *IEEE Transactions on Services Computing*, vol. 15, no. 5, pp. 2897–2909, 2021.
- [21] C. Tang, C. Zhu, N. Zhang, M. Guizani, and J. J. Rodrigues, "Sdn assisted mobile edge computing for collaborative computation offloading in industrial internet of things," *IEEE Internet of Things Journal*, 2022.
- [22] J. Chen, Q. Deng, and X. Yang, "Non-cooperative game algorithms for computation offloading in mobile edge computing environments," *Journal of Parallel and Distributed Computing*, vol. 172, pp. 18–31, 2023.
- [23] C. Lin, G. Han, X. Qi, M. Guizani, and L. Shu, "A distributed mobile fog computing scheme for mobile delay-sensitive applications in sdn-enabled vehicular networks," *IEEE Transactions on Vehicular Technology*, vol. 69, no. 5, pp. 5481–5493, 2020.
- [24] A. Lakhan, M. S. Memon, M. Elhoseny, M. A. Mohammed, M. Qabulio, M. Abdel-Basset *et al.*, "Cost-efficient mobility offloading and task scheduling for microservices iot applications in container-based fog cloud network," *Cluster Computing*, vol. 25, no. 3, pp. 2061–2083, 2022.
- [25] X. Xiong, K. Zheng, L. Lei, and L. Hou, "Resource allocation based on deep reinforcement learning in iot edge computing," *IEEE Journal on Selected Areas in Communications*, vol. 38, no. 6, pp. 1133–1146, 2020.
- [26] J. Yan, S. Bi, and Y. J. A. Zhang, "Offloading and resource allocation with general task graph in mobile edge

- computing: A deep reinforcement learning approach,” *IEEE Transactions on Wireless Communications*, vol. 19, no. 8, pp. 5404–5419, 2020.
- [27] I. M. Ali, K. M. Sallam, N. Moustafa, R. Chakraborty, M. Ryan, and K.-K. R. Choo, “An automated task scheduling model using non-dominated sorting genetic algorithm ii for fog-cloud systems,” *IEEE Transactions on Cloud Computing*, vol. 10, no. 04, pp. 2294–2308, 2022.
- [28] Y. Ye, M. Xiao, and M. Skoglund, “Mobility-aware content preference learning in decentralized caching networks,” *IEEE Transactions on Cognitive Communications and Networking*, vol. 6, no. 1, pp. 62–73, 2019.
- [29] M. K. Somesula, R. R. Rout, and D. V. Somayajulu, “Contact duration-aware cooperative cache placement using genetic algorithm for mobile edge networks,” *Computer Networks*, vol. 193, p. 108062, 2021.
- [30] Y. Liu, S. Wang, Q. Zhao, S. Du, A. Zhou, X. Ma, and F. Yang, “Dependency-aware task scheduling in vehicular edge computing,” *IEEE Internet of Things Journal*, vol. 7, no. 6, pp. 4961–4971, 2020.
- [31] Q. Shen, B.-J. Hu, and E. Xia, “Dependency-aware task offloading and service caching in vehicular edge computing,” *IEEE Transactions on Vehicular Technology*, vol. 71, no. 12, pp. 13 182–13 197, 2022.
- [32] C. Zhu, J. Tao, G. Pastor, Y. Xiao, Y. Ji, Q. Zhou, Y. Li, and A. Ylä-Jääski, “Folo: Latency and quality optimized task allocation in vehicular fog computing,” *IEEE Internet of Things Journal*, vol. 6, no. 3, pp. 4150–4161, 2018.
- [33] W. Xia, T. Q. Quek, J. Zhang, S. Jin, and H. Zhu, “Programmable hierarchical c-ran: From task scheduling to resource allocation,” *IEEE Transactions on Wireless Communications*, vol. 18, no. 3, pp. 2003–2016, 2019.
- [34] M. Abd Elaziz, L. Abualigah, and I. Attiya, “Advanced optimization technique for scheduling iot tasks in cloud-fog computing environments,” *Future Generation Computer Systems*, vol. 124, pp. 142–154, 2021.
- [35] D. Wang, D. Tan, and L. Liu, “Particle swarm optimization algorithm: an overview,” *Soft computing*, vol. 22, no. 2, pp. 387–408, 2018.
- [36] P. Liu, K. An, J. Lei, G. Zheng, Y. Sun, and W. Liu, “Scma-based multiaccess edge computing in iot systems: An energy-efficiency and latency tradeoff,” *IEEE Internet of Things Journal*, vol. 9, no. 7, pp. 4849–4862, 2021.
- [37] T. X. Tran and D. Pompili, “Joint task offloading and resource allocation for multi-server mobile-edge computing networks,” *IEEE Transactions on Vehicular Technology*, vol. 68, no. 1, pp. 856–868, 2018.
- [38] X. Li, T. Chen, D. Yuan, J. Xu, and X. Liu, “A novel graph-based computation offloading strategy for workflow applications in mobile edge computing,” *IEEE Transactions on Services Computing*, vol. 16, no. 2, pp. 845–857, 2022.
- [39] K. Poularakis and L. Tassiulas, “On the complexity of optimal content placement in hierarchical caching networks,” 2016. [Online]. Available: <https://www.dropbox.com/s/q6469v21mg2uvs7/tmccode.rar?dl=0>
- [40] M. Piorkowski, N. Sarafijanovic-Djukic, and M. Grossglauser, “CRAWDAD dataset epfl/mobility (v. 2009-02-24),” Downloaded from <https://crawdad.org/epfl/mobility/20090224/cab>, Feb. 2009, traceset: cab.
- [41] A. Karimiafshar, M. R. Hashemi, M. R. Heidarpour, and A. N. Toosi, “An energy-conservative dispatcher for fog-enabled iiot systems: when stability and timeliness matter,” *IEEE Transactions on Services Computing*, 2021.



Mekala Ratna Raju received the B.Tech. degree in Computer Science and Engineering from the QIS College of Engineering and Technology, Ongole, India, and the M.Tech. degree in Computer Science and Engineering from the National Institute of Technology Puducherry, India. He is currently pursuing the Ph.D. degree with the National Institute of Technology Tiruchirappalli, India. His primary research area includes IoT, cloud, fog and edge computing, mobile edge networks, vehicular networks, and unmanned aerial vehicles.



Sai Krishna Mothku (Member, IEEE) received the B.Tech. degree in computer science and engineering from the Kakatiya Institute of Technology and Science (KITS), Warangal, India, the M.Tech. degree in computer science and engineering-information security from the National Institute of Technology, Surathkal, India, and the Ph.D. degree in computer science and engineering from the National Institute of Technology, Warangal, India, in 2009, 2012, and 2019, respectively. Currently, he is an Assistant Professor with the Department of Computer Science and Engineering, National Institute of Technology, Tiruchirappalli, India. His research interests include wireless ad hoc networks, wireless sensor networks, IoT, cloud, fog and edge computing, mobile edge networks, vehicular networks, and unmanned aerial vehicles. Dr. Mothku is a member of IEEE Computer Society, IEEE Industrial Electronics Society and IEEE Vehicular Technology Society.



Manoj Kumar Somesula (Member, IEEE) received the B.Tech. degree in computer science and engineering from the Mahaveer Institute of Science and Technology, JNTU, Hyderabad, India, the M.Tech. degree in computer science from the School of Information Technology, JNTU, Hyderabad, India, and the Ph.D. degree in computer science and engineering from the National Institute of Technology Warangal, Warangal, India, in 2012, 2015, and 2022, respectively. He is currently an Assistant Professor with the Department of Computer Science and Engineering, Dr. B. R. Ambedkar National Institute of Technology, Jalandhar, India. He has authored or coauthored several research papers on mobile edge networks and D2D networks in reputed international journals. His research interests include edge computing, wireless networks, vehicular networks, unmanned aerial vehicles, and Internet of Things. Dr. Somesula is a member of IEEE ComSoc, IEEE Computer Society, and ACM.